

ENTERPRISE TECHNICAL TRAINING

Agentic AI

Engineering agentic systems with LangChain, LangGraph & CrewAI — for Oracle software professionals, senior engineers, developers, data engineers, ML engineers, cloud engineers, and solution architects.

COURSE AGENDA

1

Foundations of Agentic AI

2

Reasoning, Planning & Tool Use

3

Memory, RAG & Multi-Agent Systems

4

LangChain Framework

5

LangGraph Framework

6

CrewAI Framework

7

Framework Comparison & Selection

8

Real-World Use Cases, Safety & Deployment

9

Capstone Project

MODULE 1

Foundations of Agentic AI

LEARNING OBJECTIVES

- Define Agentic AI precisely and distinguish it from a bare LLM call
- Explain the reason → act → observe loop and what 'autonomy' actually means in practice
- Differentiate Agents from Chatbots and RAG systems on concrete engineering dimensions
- Identify the three foundational agent architecture patterns: Single Agent, Multi-Agent, Supervisor/Worker
- Build a minimal working tool-calling agent loop

What Is Agentic AI?

DEFINITION

Agentic AI refers to AI systems that autonomously pursue multi-step goals by reasoning about what to do next, selecting and invoking tools or actions, observing the results, and adapting their next step — rather than producing a single one-shot response to a prompt. The defining trait is a closed control loop: the system decides its own next action based on the outcome of its previous one.

EXAMPLE

Prompt: "Resolve ticket #4471: customer says their invoice total looks wrong."

A bare LLM would guess an answer from the ticket text alone. An agentic system instead decides to query the billing database, compares the line items to the customer's contract terms, and only then drafts a response — choosing each of those steps itself.

REAL-TIME ENTERPRISE USE CASE

An accounts-payable agent at a manufacturing firm autonomously matches incoming vendor invoices against purchase orders and goods-receipt records, flags mismatches above a tolerance threshold, and routes only the exceptions to a human approver — reducing manual three-way-match effort by the majority of routine invoices.

LLM vs. Agent: Reasoning, Action & Autonomy

DEFINITION

An LLM alone maps an input prompt to an output in a single forward pass — it has no memory of the live external world beyond what's in its context window, and it cannot take action. An agent wraps an LLM in a control loop that alternates between reasoning (deciding what to do next), acting (invoking a tool or function against a real system), and observing (reading that tool's result) — repeating until the goal is satisfied or a stop condition is hit. Autonomy is a spectrum: how many of those loop iterations happen without a human approving each step.

EXAMPLE

Prompt: "What's the current inventory level for SKU-2291, and should we reorder?"

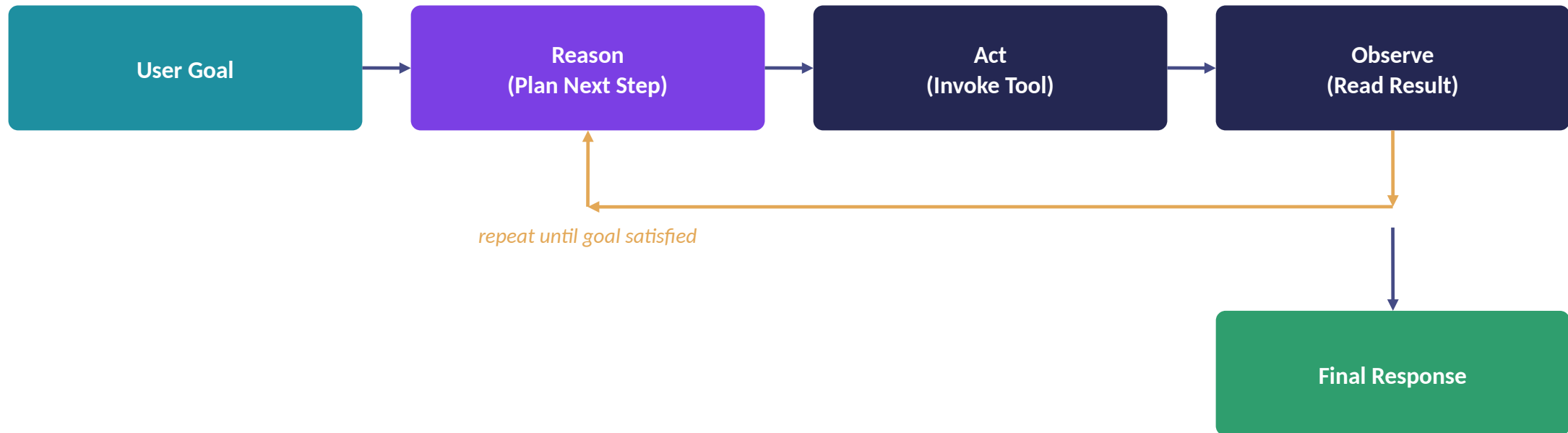
A bare LLM will produce a plausible-sounding but ungrounded number. An agent instead calls an inventory-lookup tool, observes the real returned quantity and reorder threshold, then reasons over that real data before answering — its answer is only as good as the loop that produced it.

REAL-TIME ENTERPRISE USE CASE

A demand-planning agent checks live inventory and open purchase orders every morning, reasons about which SKUs will stock out before the next replenishment cycle, and either raises a purchase requisition automatically (low-risk items) or flags it for planner review (high-value items) — autonomy is deliberately capped by item risk class.

The Agent Control Loop

Goal → Reason → Act (Tool Call) → Observe → loop until goal satisfied → Final Response



The loop is what separates an agent from a single-shot LLM call: each cycle's observation informs the next reasoning step.

The Agent Control Loop — Explained

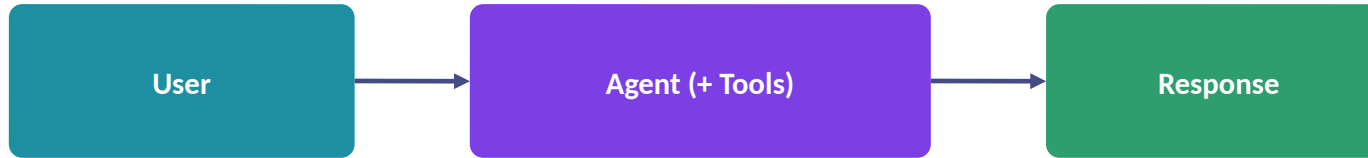
1. User Goal: the agent receives a natural-language objective, not a fixed script of steps.
2. Reason: the LLM decides what the single next action should be, given the goal and everything observed so far.
3. Act: the agent invokes a real tool — an API call, a database query, code execution — against a live system.
4. Observe: the tool's actual result (not a guess) is read back into the agent's working context.
5. Loop: the agent reasons again over the new observation — this repeats as many times as the goal requires.
6. Final Response: once the agent determines the goal is satisfied (or a stop condition is hit), it returns a grounded answer.

Chatbots vs. RAG Systems vs. Agents

Dimension	Chatbot	RAG System	Agent
Core capability	Turn-based conversational response	Retrieval-grounded response over a knowledge base	Autonomous multi-step task execution
Decision-making	None beyond dialogue/intent routing	None — always retrieve, then generate	Plans, selects tools, and adapts across steps
External actions	Rare, and only if hand-scripted	Read-only retrieval from a vector/document store	Reads and writes — calls APIs, runs code, updates systems
Statefulness	Session-level conversation history	Stateless per query unless paired with an agent	Maintains goal state, memory, and intermediate results
Typical failure mode	Repeats a canned or off-topic response	Hallucinates when retrieval misses relevant context	Can loop indefinitely or take an unintended action without guardrails
Typical enterprise use	FAQ bot, order-status lookup	Internal policy/document Q&A	Incident response, research synthesis, coding assistance

Agentic AI Architecture Patterns

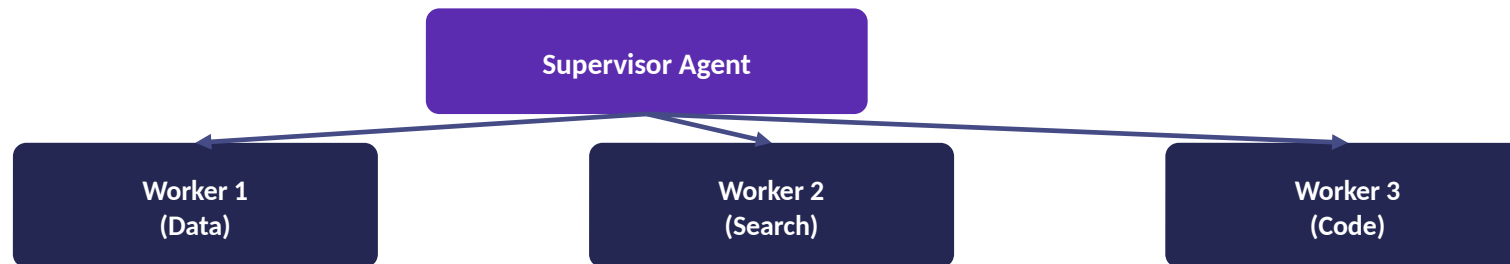
Single Agent



Multi-Agent
(Peer-to-Peer)



Supervisor /
Worker



Single Agent handles one bounded goal end-to-end; Multi-Agent (Peer) splits work across specialists that message each other directly; Supervisor/Worker centralizes planning in one agent that delegates to, and consolidates results from, subordinate workers.

Agentic AI Architecture Patterns — Explained

1. Single Agent: one agent owns the entire goal end-to-end, calling whatever tools it needs — the simplest pattern, and the right default until complexity proves otherwise.
2. Multi-Agent (Peer-to-Peer): specialized agents (research, analysis, writing) exchange messages directly and collaboratively converge on a result — flexible, but harder to make predictable at scale.
3. Supervisor/Worker: a single supervisor agent decomposes the goal, delegates bounded sub-tasks to worker agents, and consolidates their results — the pattern most enterprise systems converge on because it keeps control and auditability centralized.
4. Choosing among them is a Module 7 topic (framework selection) — but the architectural decision should always come before the framework decision, not after.

Hands-On: A Minimal Calculator Agent

```
def calculator(expression: str) -> str:
    return str(eval(expression, {"__builtins__": {}}))

tools = [{
    "name": "calculator",
    "description": "Evaluate a basic arithmetic expression",
    "parameters": {"type": "object",
        "properties": {"expression": {"type": "string"}}}
}]

messages = [{"role": "user",
    "content": "What is (482 * 17) - 96, divided by 2?"}]
response = chat_client.chat(messages=messages, tools=tools)

while response.tool_calls:
    for call in response.tool_calls:
        result = calculator(call.arguments["expression"])
        messages.append({"role": "tool", "name": call.name,
            "content": result})
        response = chat_client.chat(messages=messages, tools=tools)

print(response.content) # grounded, arithmetically correct answer
```

EXPECTED OUTPUT

The agent calls the calculator tool with `"(482 * 17) - 96) / 2"`, observes 4057.0, and returns: "The result is 4057." — correct even though LLMs are notoriously unreliable at raw arithmetic.

ENTERPRISE USE CASE

This is the smallest possible instance of the agent loop from the previous diagram: Reason (decide a tool is needed) → Act (call calculator) → Observe (read the result) → Final Response. Every framework in Modules 4–6 is, underneath, a more structured way of writing this same loop.

MODULE 2

Reasoning, Planning & Tool Use

LEARNING OBJECTIVES

- Explain the ReAct pattern and why interleaving reasoning with real observations reduces hallucination
- Compare Chain-of-Thought, Tree-of-Thought, and Graph-of-Thought reasoning strategies
- Design a formal tool/function schema and wire it into an LLM's tool-calling loop
- Write a production-grade ReAct prompt template
- Apply reflection patterns to catch and correct an agent's own mistakes

ReAct: Reason + Act

DEFINITION

ReAct is a prompting pattern that interleaves explicit natural-language reasoning traces ("Thought: ...") with concrete actions ("Action: ...") and their real results ("Observation: ...") inside one continuous transcript, repeating until the agent emits a final answer. Externalizing the reasoning and grounding each step in a real observation — rather than reasoning the whole way through on recalled knowledge alone — measurably reduces hallucination.

EXAMPLE

Question: "What is the population of the capital of France?"

Thought: I need the capital of France first, then its population.

Action: search["capital of France"] → Observation: "Paris"

Thought: Now I need Paris's population.

Action: search["population of Paris"] → Observation: "~2.1 million (city proper)"

Thought: I have enough information.

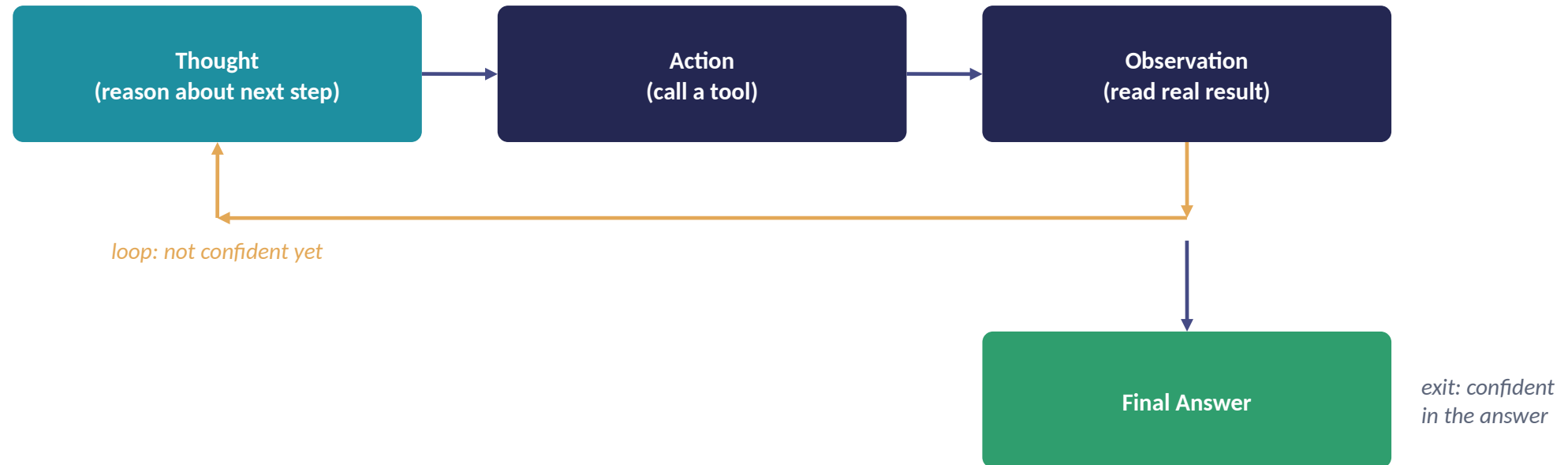
Final Answer: Paris has a population of approximately 2.1 million.

REAL-TIME ENTERPRISE USE CASE

A compliance-research agent answering "Does our EU customer data-retention policy meet GDPR Article 17?" interleaves searches of the internal policy repository with searches of the regulation text, reasoning explicitly at each step — producing an auditable, line-by-line trace a compliance officer can review, not just an opaque final answer.

The ReAct Loop

Thought → Action → Observation → loop until confident → Final Answer



Each Thought is conditioned on the most recent real Observation — the model is never allowed to reason more than one uncontested step ahead of the facts.

The ReAct Loop — Explained

1. Thought: the model writes out, in natural language, what it currently believes and what it needs to find out next.
2. Action: the model emits a structured call to one available tool, with concrete arguments — not a vague intention.
3. Observation: the tool executes against a real system and its literal result is appended to the transcript.
4. Loop: the next Thought is generated conditioned on that real Observation, not on what the model merely expects the result to be.
5. Final Answer: once the model judges it has sufficient grounded evidence, it exits the loop with a direct answer — the exit condition is model-judged, so prompt design (next topic) and guardrails (Module 8) both matter.

Chain-, Tree- & Graph-of-Thought

Pattern	Reasoning Shape	When It Helps	Cost
Chain-of-Thought (CoT)	One linear sequence of intermediate reasoning steps, committed to as it goes	Multi-step arithmetic, straightforward multi-hop lookups — problems with one clear solution path	Lowest — a single pass
Tree-of-Thought (ToT)	Branches into several candidate next-steps at each point, evaluates them, prunes weak branches, backtracks	Problems with several plausible approaches where an early wrong turn is costly to discover late (planning, puzzle-like tasks)	Higher — multiple candidate generations plus an evaluation step per branch
Graph-of-Thought (GoT)	Reasoning steps form a graph — branches can merge, and one step can depend on several prior steps combined	Synthesis tasks where partial results from different angles must be combined into one conclusion (e.g., multi-source research)	Highest — requires explicit dependency tracking between steps

In production, start with CoT/ReAct — reach for ToT or GoT only when a measured failure mode (bad early commitments, or unmerged partial evidence) justifies the added latency and cost.

Advanced Planning: Tree-of-Thought in Practice

DEFINITION

Tree-of-Thought generates several candidate next-steps at a decision point, uses the model itself (or a separate evaluator call) to score each candidate's promise, keeps the strongest, and can backtrack if a branch later proves unproductive — trading extra inference calls for a materially lower rate of committing early to a bad plan.

EXAMPLE

Task: "Design a database migration plan from Oracle 19c to Oracle Database 23ai with zero downtime."

Branch A: Data Guard switchover approach. Branch B: GoldenGate replication approach. Branch C: export/import during a maintenance window.

The agent scores each branch against the zero-downtime constraint, discards Branch C immediately, and explores A and B in more depth before committing.

REAL-TIME ENTERPRISE USE CASE

A cloud-migration planning agent evaluates three candidate re-platforming strategies for a legacy application in parallel branches, scoring each on cost, risk, and downtime before presenting the architect with the single best-scored plan and a summary of why the alternatives were rejected — turning a black-box recommendation into an auditable comparison.

Tool Use & API Calling

DEFINITION

Tool (function) calling is the formal mechanism by which a developer describes available actions to an LLM — each with a name, a natural-language description, and a JSON-schema parameter signature — so the model can emit a structured request to invoke one, with arguments it has inferred from the conversation. The application (never the model) actually executes the call against a real system and returns the result as an observation.

EXAMPLE

```
{
  "name": "get_order_status",
  "description": "Look up the current status of a customer order",
  "parameters": {
    "type": "object",
    "properties": {
      "order_id": {"type": "string", "description": "e.g. ORD-48213"}
    },
    "required": ["order_id"]
  }
}
```

REAL-TIME ENTERPRISE USE CASE

An order-support agent is given `get_order_status`, `check_return_eligibility`, and `issue_refund` as tools. It autonomously chains the first two before ever considering the third — `issue_refund` is scoped in IAM to a spending cap, so even an over-eager agent cannot exceed a bounded blast radius.

A Production ReAct Prompt Template

SYSTEM:

You solve tasks by reasoning step by step. At each turn, output EXACTLY ONE of the following, then stop and wait:

Thought: <your reasoning about what to do next>

Action: <tool_name>[<tool_input>]

Final Answer: <your final answer, only once you are confident>

Never fabricate an Observation — it will be supplied by the tool.
If a tool call fails, reason about why before retrying.

Available tools:

- search[query]: searches internal knowledge base
- sql_query[query]: runs a read-only SQL query
- calculator[expression]: evaluates a math expression

Question: {input}

WHY THIS TEMPLATE WORKS

Forcing exactly one Thought/Action pair per turn (rather than letting the model plan several steps ahead in one message) keeps every intermediate belief checkable against a real Observation before the next one is formed.

ENTERPRISE USE CASE

This is the literal system prompt underlying OCI Generative AI Agents' ReAct-style tool orchestration and LangChain's `create_react_agent` — frameworks in Modules 4–6 automate the transcript bookkeeping, but the underlying contract is exactly this template.

Reflection Patterns

DEFINITION

Reflection adds a self-critique step after an agent produces a candidate output: a second pass (often the same model, sometimes a dedicated critic prompt) evaluates that output against the original goal and explicit criteria, and either approves it or sends it back with specific feedback for revision — closing the loop between 'produced an answer' and 'verified the answer is actually good.'

EXAMPLE

An agent drafts SQL for "top 10 customers by revenue this quarter." A reflection pass checks: does the query filter on the correct date range, exclude refunds, and match the schema's actual column names? It catches that the draft used `order_date` instead of the schema's `invoice_date`, and sends it back for one correction before returning the final query.

REAL-TIME ENTERPRISE USE CASE

A code-review agent drafts a bug fix, then reflects against the original ticket, the test suite, and the repository's style guide before opening a pull request — catching an incomplete fix (missed edge case) in roughly a third of drafts before a human ever sees them, per internal measurement at teams that have adopted this pattern.

Hands-On: A Search + Python Tool Agent

```
tools = [
    {"name": "web_search", "description": "Search the web for a query",
     "parameters": {"type": "object",
                    "properties": {"query": {"type": "string"}}}},
    {"name": "python_exec", "description": "Run a Python snippet, return stdout",
     "parameters": {"type": "object",
                    "properties": {"code": {"type": "string"}}}}
]

messages = [{"role": "user", "content":
    "Find the current USD/EUR exchange rate and compute what "
    "$18,500 in quarterly SaaS spend equals in EUR."}]

response = chat_client.chat(messages=messages, tools=tools)
while response.tool_calls:
    for call in response.tool_calls:
        result = dispatch(call.name, call.arguments) # routes to
                                                    # web_search or python_exec
        messages.append({"role": "tool", "name": call.name, "content": result})
    response = chat_client.chat(messages=messages, tools=tools)

print(response.content)
```

EXPECTED OUTPUT

The agent calls `web_search` for the live rate, then `python_exec` to multiply — combining a real-time fact with a real computation, neither of which an LLM can be trusted to produce from memory alone.

ENTERPRISE USE CASE

This two-tool pattern (one fact-finding tool, one computation tool) generalizes directly to enterprise FinOps agents that pull live cloud-spend data and compute forecasted month-end totals against budget.

Hands-On: Implement a ReAct Prompt Template

```

REACT_TEMPLATE = """You solve tasks step by step.
At each turn output exactly one line:
Thought: <reasoning>
Action: <tool>[<input>]
or, once confident:
Final Answer: <answer>

Tools available: {tool_names}

Question: {question}
{scratchpad}"""

def run_react(question, tools, max_steps=6):
    scratchpad = ""
    for step in range(max_steps):
        prompt = REACT_TEMPLATE.format(
            tool_names=", ".join(t["name"] for t in tools),
            question=question, scratchpad=scratchpad)
        text = llm.complete(prompt, stop=["Observation:"])
        if "Final Answer:" in text:
            return text.split("Final Answer:")[1].strip()
        action, tool_input = parse_action(text) # e.g. search["query"]
        observation = dispatch(action, tool_input)
        scratchpad += text + f"\nObservation: {observation}\n"
    return "Max steps reached without a confident answer."

```

EXPECTED OUTPUT

A hand-rolled ReAct loop that builds the scratchpad turn by turn and stops generation at "Observation:" so the model never hallucinates a tool result — the exact mechanism frameworks like LangChain wrap for you.

ENTERPRISE USE CASE

Building this by hand once is deliberate: it demystifies what `create_react_agent()` in Module 4 is actually doing under the hood, which makes debugging a framework-based agent in production far less of a black box.

MODULE 3

Memory, RAG & Multi-Agent Systems

LEARNING OBJECTIVES

- Differentiate short-term, long-term, and episodic memory and where each is implemented
- Explain how Retrieval-Augmented Generation grounds an agent's responses in enterprise data
- Design a Planner / Solver / Reviewer multi-agent role split
- Build a working Planner-Worker system

Short-Term, Long-Term & Episodic Memory

Memory Type	What It Holds	Typical Implementation	Example
Short-term (working)	The current task's conversation and scratchpad — everything visible in this run	Message history / ReAct transcript passed in the context window on every call	The Thought/Action/Observation trace built up during one incident investigation
Long-term	Durable facts and preferences that persist across sessions and users	Vector store or key-value store queried at the start of a new session	Recalling a user's stated preferred deployment region across unrelated chats, weeks apart
Episodic	Records of specific past task episodes — what was tried, and what happened	A log of past transcripts, retrievable by similarity to the current situation	Recalling that a similar service outage three weeks ago was resolved by a config rollback, and trying that first

Long-Term Memory via Vector Stores

DEFINITION

Long-term memory is typically implemented by embedding salient facts (user preferences, past decisions, entity resolutions) into a vector store at the end of a session, then retrieving the top-k most relevant memories at the start of a new one and injecting them into the agent's context — the same retrieval mechanism RAG uses over documents, applied instead to the agent's own history.

EXAMPLE

After a session where a user states "Always deploy our staging environment in ap-mumbai-1," that fact is embedded and stored. Three weeks later, in an unrelated session, the user asks the agent to provision a new staging service — the agent retrieves that stored preference and defaults the region without being asked again.

REAL-TIME ENTERPRISE USE CASE

A DevOps assistant remembers, across sessions and without being told twice, that a given team's production changes require a two-person approval — retrieved automatically whenever that team is mentioned, and enforced as a guardrail rather than merely recalled as trivia.

RAG in Agents

DEFINITION

Retrieval-Augmented Generation equips an agent with a retrieval tool over a vector database of enterprise documents: the agent embeds a query, retrieves the most semantically similar passages, and conditions its response on that retrieved context — grounding answers in current, citable enterprise content instead of only the model's training data. In an agentic (as opposed to fixed-pipeline) RAG system, retrieval is a tool the agent chooses to invoke, potentially more than once, rather than a mandatory first step.

EXAMPLE

Question: "What is our current policy on third-party data processors for EU customers?"

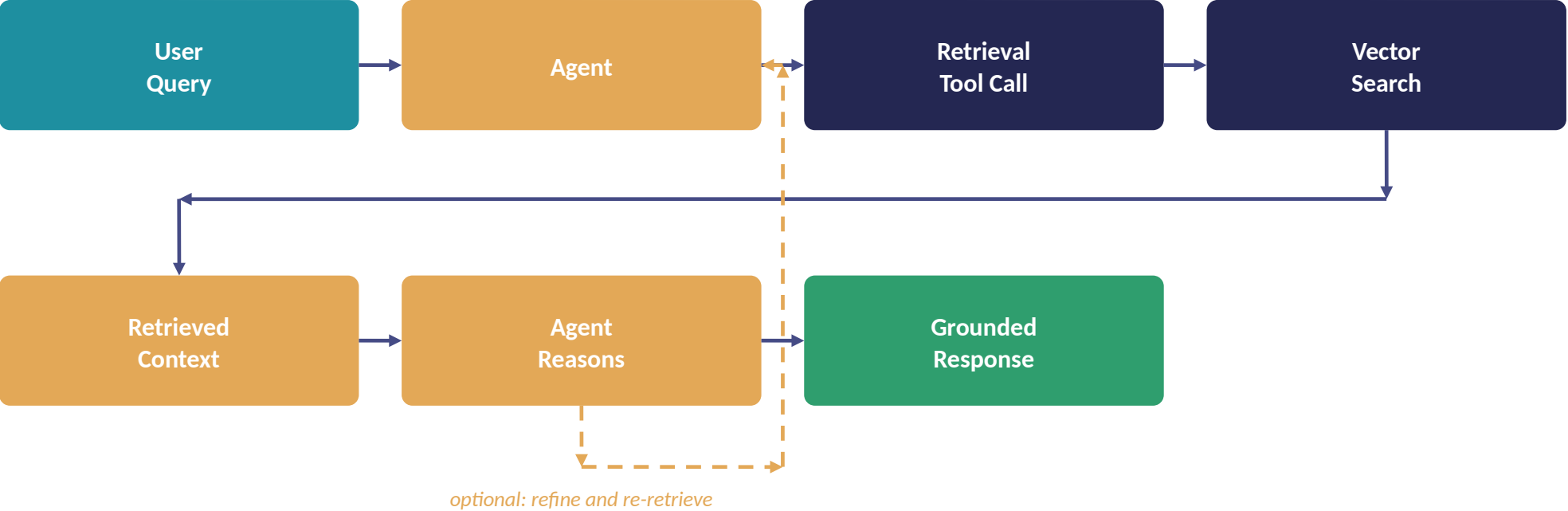
The agent embeds the question, retrieves the three most similar passages from the indexed policy corpus (e.g., stored with Oracle Database 23ai's native AI Vector Search alongside the relational policy metadata), and answers using only those retrieved passages — citing the source section.

REAL-TIME ENTERPRISE USE CASE

An enterprise knowledge-management agent indexes HR policies, engineering runbooks, and product documentation into one vector store. When asked a cross-domain question, it retrieves from all three sources, reasons over the combined context, and cites which document each claim came from — turning scattered institutional knowledge into a single, auditable answer surface.

Agentic RAG Flow

User Query → Agent → Retrieval Tool → Vector Search → Retrieved Context → Agent Reasons → Grounded Response



Because retrieval is a tool call the agent decides to make (not a fixed pipeline step), the agent can re-query with a refined search if the first retrieval is insufficient.

Agentic RAG Flow — Explained

1. User Query: the agent receives a natural-language question that may or may not require retrieval.
2. Agent: reasons about whether it needs enterprise-specific grounding, and if so, invokes the retrieval tool.
3. Vector Search: the query is embedded and matched against the indexed document store using vector similarity.
4. Retrieved Context: the top-k most relevant passages are returned as an observation, with source metadata attached.
5. Agent Reasons: the agent conditions its next Thought on the retrieved passages rather than on recalled training data.
6. Grounded Response: the final answer cites the retrieved passages — and if the context was insufficient, the agent can re-query with a refined search instead of answering ungrounded.

Planner, Solver & Reviewer Roles

DEFINITION

A common multi-agent role split separates planning from execution from verification: a Planner agent decomposes a goal into an ordered set of subtasks, one or more Solver agents execute those subtasks (each potentially with its own tools), and a Reviewer agent checks the combined output against the original goal before it is released — rejecting and sending work back to the Planner if it falls short.

EXAMPLE

Goal: "Produce a competitive pricing analysis for our top 3 cloud database competitors."

Planner: splits into "research competitor A pricing," "research competitor B pricing," "research competitor C pricing," "synthesize into a comparison."

Solvers: three run in parallel on the research subtasks; a fourth performs synthesis.

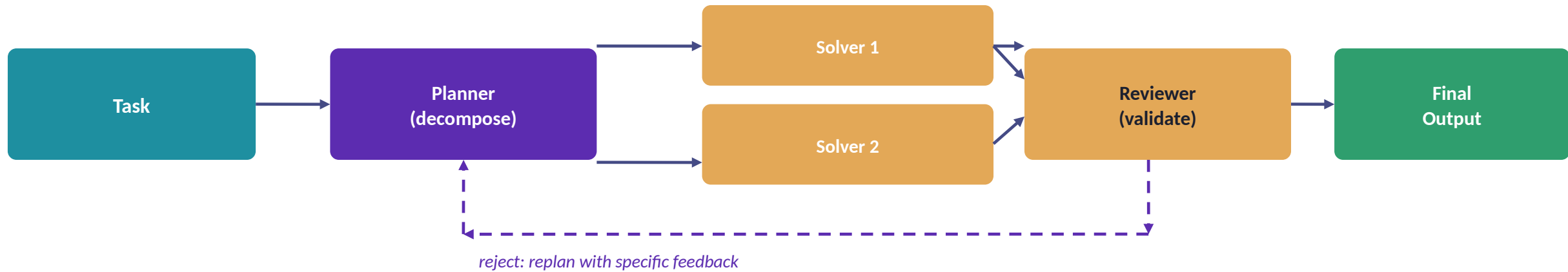
Reviewer: checks that all three competitors are covered and pricing is dated within the last quarter — rejects and asks the Planner to re-run competitor B's research, which used a stale source.

REAL-TIME ENTERPRISE USE CASE

A quarterly-report generation system uses exactly this split: a Planner enumerates the required sections, Solver agents pull and draft each section from live data sources, and a Reviewer agent checks numeric consistency across sections (e.g., that a total in the summary matches the sum of the detail tables) before the report is released to finance.

Planner / Solver / Reviewer Workflow

Task → Planner (decompose) → Solver Agent(s) (execute) → Reviewer (validate) → Final Output



A Reviewer rejection routes back to the Planner with specific feedback — not back to the Solver directly — so the plan itself, not just the last execution attempt, can be revised.

Hands-On: A Planner–Worker System

```
def plan(goal: str) -> list[str]:
    """LLM call: decompose the goal into an ordered subtask list."""
    resp = llm.complete(PLANNER_PROMPT.format(goal=goal))
    return parse_subtasks(resp)          # e.g. ["research A", "research B", ...]

def solve(subtask: str) -> str:
    """Worker agent: run the ReAct loop from Module 2 on one subtask."""
    return run_react(subtask, tools=research_tools)

def review(goal: str, results: list[str]) -> tuple[bool, str]:
    """LLM call: does the combined output satisfy the goal?"""
    resp = llm.complete(REVIEWER_PROMPT.format(goal=goal, results=results))
    return parse_verdict(resp)          # (approved: bool, feedback: str)

def run_planner_worker(goal, max_replans=2):
    for attempt in range(max_replans + 1):
        subtasks = plan(goal)
        results = [solve(t) for t in subtasks]      # parallelizable
        approved, feedback = review(goal, results)
        if approved:
            return synthesize(results)
        goal = f"{goal}\n\nReviewer feedback: {feedback}"
    return "Escalated to human: could not satisfy reviewer after retries."
```

EXPECTED OUTPUT

plan() returns 3 subtasks, solve() runs each as an independent ReAct agent, review() catches that subtask 2's source was outdated, and one replan cycle produces an approved synthesis.

ENTERPRISE USE CASE

This exact skeleton — plan / parallel-solve / review / conditional replan — is what LangGraph (Module 5) formalizes as a graph with a conditional edge, and what CrewAI (Module 6) formalizes as a hierarchical process; seeing it in plain Python first makes both frameworks' abstractions legible rather than magical.

MODULE 4

Agent Frameworks: LangChain

LEARNING OBJECTIVES

- Identify LangChain's core abstractions: Chains, Agents, Tools, and Memory
- Explain how AgentExecutor drives the reason/act/observe loop under the hood
- Build a working tool-calling LangChain agent
- Map a LangChain agent onto a concrete Oracle enterprise use case

LangChain: Core Abstractions

DEFINITION

LangChain is a composition framework for LLM applications built from four core abstractions: Chains (fixed sequences of LLM/tool calls wired together declaratively), Tools (typed functions the LLM can invoke, each with a name/description/schema), Memory (a pluggable store of prior conversation or state, injected into the prompt), and Agents (an LLM bound to a set of tools and a prompt, whose tool choice is executed and re-fed by an AgentExecutor loop). Chains give you a fixed pipeline; Agents give you a dynamic, LLM-directed one.

EXAMPLE

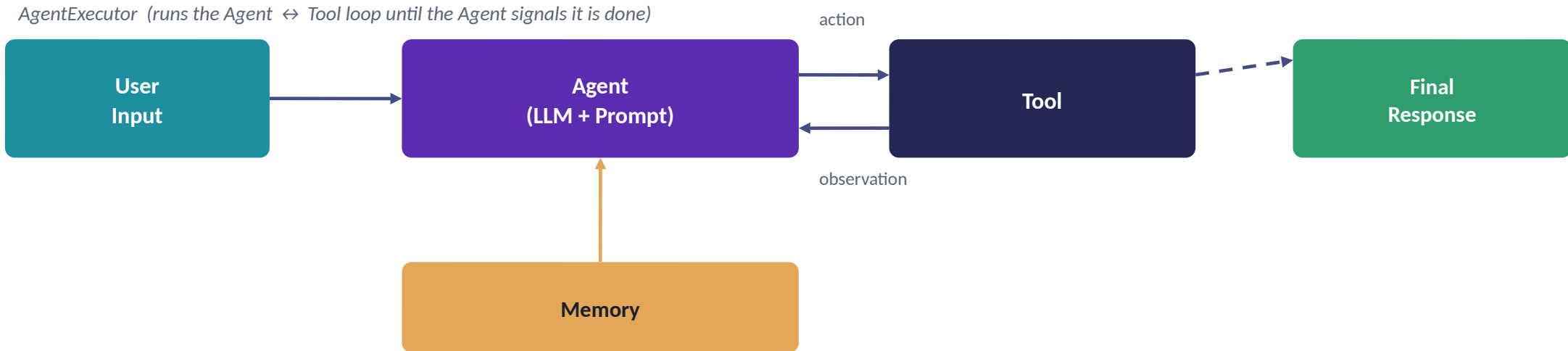
A support Chain might be fixed: `classify_intent` → `retrieve_docs` → `draft_reply`, always in that order. A support Agent instead decides for itself, per ticket, whether it needs to retrieve documentation, check order status, or both — the sequence is not hard-coded, the LLM's tool choice determines it turn by turn.

REAL-TIME ENTERPRISE USE CASE

An Oracle DBA-support tool wraps a schema-lookup Tool and a read-only SQL-execution Tool inside a LangChain Agent with ConversationBufferMemory — engineers ask follow-up questions across a session ("now break that down by region") and the agent retains enough context to refine its own prior query rather than starting over.

LangChain Agent Architecture

AgentExecutor drives the loop: Agent reasons over Prompt + Memory, chooses a Tool, executes it, and feeds the result back in.



The Agent itself never executes a tool — AgentExecutor is the runtime that parses the Agent's chosen action, actually calls the Tool, and appends the observation before invoking the Agent again.

LangChain Agent Architecture — Explained

1. User Input enters `AgentExecutor.invoke()` as the task the agent must accomplish.
2. The Agent (an LLM bound to a prompt template and a tool list) reasons over the input plus anything in Memory, and emits either a tool call or a final answer.
3. `AgentExecutor` parses that output: if it's a tool call, it actually invokes the named Tool with the Agent's chosen arguments.
4. The Tool's real return value is appended to the transcript as an observation, and the Agent is invoked again with that new context.
5. Memory (e.g., `ConversationBufferMemory` or a message-history store) is read at the start of each Agent call and updated at the end, so state persists across turns, not just within one `AgentExecutor` run.
6. Once the Agent's output contains no further tool call, `AgentExecutor` returns the Final Response to the caller.

Building a Tool-Calling LangChain Agent

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.tools import tool
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

@tool
def get_order_status(order_id: str) -> str:
    """Look up the current status of a customer order by its ID."""
    return db.lookup_order(order_id).status

tools = [get_order_status]
llm = ChatOpenAI(model="gpt-4o", temperature=0)

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful order-support assistant."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])

agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = executor.invoke({"input": "What's the status of order ORD-48213?"})
print(result["output"])
```

EXPECTED OUTPUT

AgentExecutor logs: calling get_order_status(order_id="ORD-48213") →
"Shipped, arriving Sep 24" → final output: "Order ORD-48213 has shipped and
is arriving September 24."

ENTERPRISE USE CASE

The @tool decorator auto-generates the JSON schema LangChain sends to the model from the function's type hints and docstring — the same tool-calling contract from Module 2, with LangChain handling the schema generation and transcript bookkeeping for you.

LangChain in Practice: Oracle SQL Assistant

▪ Business Problem:

Business analysts need ad-hoc answers from a reporting database but don't write SQL, so every question becomes a ticket in the data-engineering team's queue.

▪ AI Solution:

A LangChain agent wraps a schema-introspection tool and a read-only, row-limited SQL-execution tool behind a natural-language chat interface.

▪ Architecture:

Analyst question → LangChain Agent (schema + SQL tools, ConversationBufferMemory) → AgentExecutor loop → generated SQL executed read-only → tabular result → natural-language summary.

▪ Frameworks Used:

LangChain (create_tool_calling_agent, AgentExecutor, ConversationBufferMemory).

EXAMPLE SCENARIO

"Of our top 10 customers by revenue this quarter, which ones had a support ticket in the last 30 days?"

EXPECTED RESULT

A correctly-scoped SQL query joining the revenue and ticketing tables, executed read-only, returned as a formatted table plus a one-line summary — with the generated SQL shown for the analyst to verify.

SAFETY CONSIDERATIONS

The SQL-execution tool is a database role with SELECT-only grants and a hard row/time-out limit; the agent can never be tricked into a write, regardless of how the prompt is phrased.

MODULE 5

Agent Frameworks: LangGraph

LEARNING OBJECTIVES

- Explain LangGraph's graph-of-nodes model: State, Nodes, Edges, and Conditional Edges
- Contrast LangGraph's explicit control flow with LangChain's AgentExecutor loop
- Build a ReAct agent as an explicit, cyclic state graph
- Apply LangGraph's checkpointing to support durable, human-in-the-loop workflows

LangGraph: Agents as State Graphs

DEFINITION

LangGraph models an agent as an explicit graph: a shared, typed State object flows through Nodes (plain functions that read and update state), connected by Edges that can be fixed or Conditional (a routing function decides which node runs next, based on the current state). Unlike AgentExecutor's implicit internal loop, the reason/act/observe cycle in LangGraph is a visible cycle in the graph itself — you can see, and directly control, every possible transition.

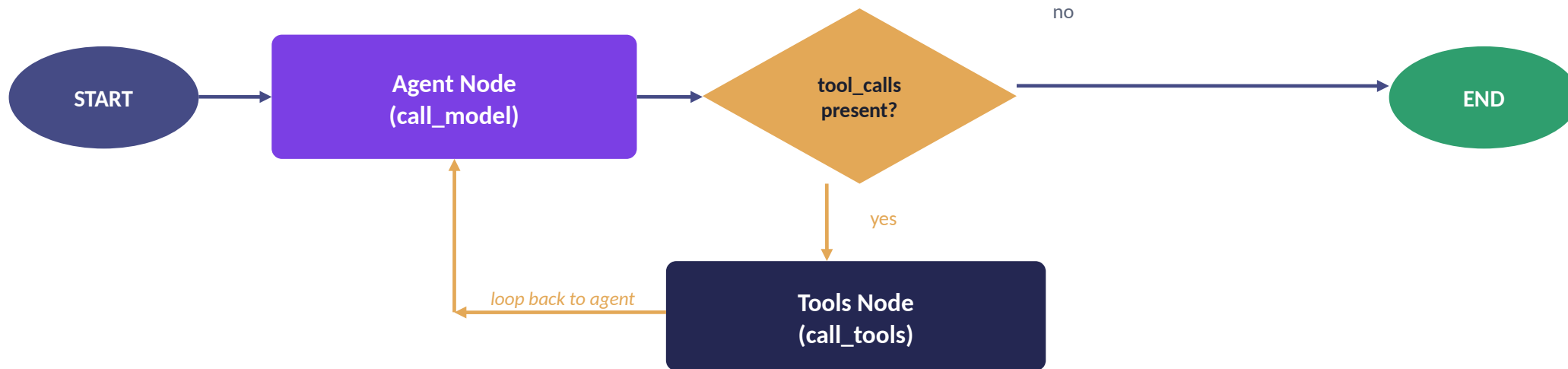
EXAMPLE

A support-triage graph has an agent node, a tools node, and a conditional edge that checks: did the last model output request a tool call? If yes, route to the tools node, then back to the agent node — an explicit cycle. If no, route to END. The exact same ReAct loop from Module 2, now expressed as graph topology instead of implicit control flow.

REAL-TIME ENTERPRISE USE CASE

A multi-step change-approval workflow is modeled as a graph with an explicit human-approval node: the graph runs up to that node, then pauses (an 'interrupt') until a human approves in a separate system call — LangGraph's checkpointing persists the paused state durably, so approval can arrive minutes or days later without losing progress.

A ReAct Agent as a LangGraph State Graph



The conditional edge is the graph's only decision point — every other transition is a fixed edge — which is what makes a LangGraph agent's possible paths fully enumerable and testable.

A ReAct Agent as a LangGraph State Graph — Explained

1. START enters the graph with the initial State — typically `{"messages": [user_input]}`.
2. The Agent Node calls the LLM (bound to tools) on the current message state and appends its response, which may or may not include tool calls.
3. A Conditional Edge inspects the latest message: if it contains tool calls, route to the Tools Node; otherwise route to END.
4. The Tools Node executes each requested tool against real systems and appends the results as new messages to the shared State.
5. A fixed edge routes from the Tools Node straight back to the Agent Node — closing the cycle with the new observations now in state.
6. The graph keeps cycling through Agent → Conditional Edge → Tools → Agent until the conditional edge routes to END, at which point the final state is returned.

Building the Graph with LangGraph

```

from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

class AgentState(TypedDict):
    messages: Annotated[list, operator.add]

def call_model(state: AgentState):
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}
def call_tools(state: AgentState):
    last = state["messages"][-1]
    return {"messages": [execute_tool(tc) for tc in last.tool_calls]}
def should_continue(state: AgentState):
    return "tools" if state["messages"][-1].tool_calls else END

graph = StateGraph(AgentState)
graph.add_node("agent", call_model)
graph.add_node("tools", call_tools)
graph.add_edge(START, "agent")
graph.add_conditional_edges("agent", should_continue, {"tools": "tools", END: END})
graph.add_edge("tools", "agent")

app = graph.compile(checkpointer=MemorySaver())
result = app.invoke({"messages": [("user", "Check inventory for SKU-2291)]}],
                    config={"configurable": {"thread_id": "session-1"}})

```

EXPECTED OUTPUT

app.invoke cycles Agent → Tools → Agent once (an inventory-lookup call, then a grounded summary), then routes to END — thread_id lets the same conversation resume later from the checkpointer.

ENTERPRISE USE CASE

The compiled graph is a durable object: MemorySaver (or a production checkpointer) persists state at every node transition — what makes the human-in-the-loop interrupt on the next slide possible.

LangGraph in Practice: Change-Approval Workflow

▪ Business Problem:

Infrastructure change requests require a documented human approval step, but the diagnostic and drafting work around that approval is manual and slow.

▪ AI Solution:

A LangGraph state graph automates diagnosis and change-plan drafting, then pauses at an explicit human-approval node until an approver responds — potentially hours later.

▪ Architecture:

Alert → Diagnose Node → Draft-Plan Node → Human-Approval Node (interrupt) → [approved] Execute Node → Notify Node → END.

▪ Frameworks Used:

LangGraph (StateGraph, add_conditional_edges, checkpointer-backed interrupt).

EXAMPLE SCENARIO

```
"A P2 alert fired for the payments API.  
Diagnose  
root cause and draft a remediation plan for  
approval."
```

EXPECTED RESULT

A drafted remediation plan sits durably paused at the approval node; when an approver calls the resume API hours later, the graph continues from exactly that point — no re-diagnosis, no lost context.

SAFETY CONSIDERATIONS

The Execute Node is unreachable in the graph topology except via the Human-Approval Node — the workflow's own structure, not a prompt instruction, is what prevents unapproved changes from running.

MODULE 6

Agent Frameworks: CrewAI

LEARNING OBJECTIVES

- Explain CrewAI's role-based abstractions: Agent, Task, Crew, and Process
- Contrast Sequential and Hierarchical process modes
- Build a working multi-agent Crew with task handoff
- Map a Crew onto a concrete enterprise report-generation use case

CrewAI: Role-Based Multi-Agent Collaboration

DEFINITION

CrewAI orchestrates multiple agents around explicit roles rather than a single agent's tool list. Each Agent is defined by a role, goal, and backstory (which shape its persona and reasoning focus) and can hold its own tools. Each Task has a description, an expected output, and an assigned Agent, and can declare other tasks as context so later agents see earlier agents' results. A Crew bundles agents and tasks under a Process — Sequential (tasks run in a fixed order, each handed the prior task's output) or Hierarchical (a manager agent dynamically delegates tasks and reviews results, similar to Module 3's Planner/Reviewer pattern).

EXAMPLE

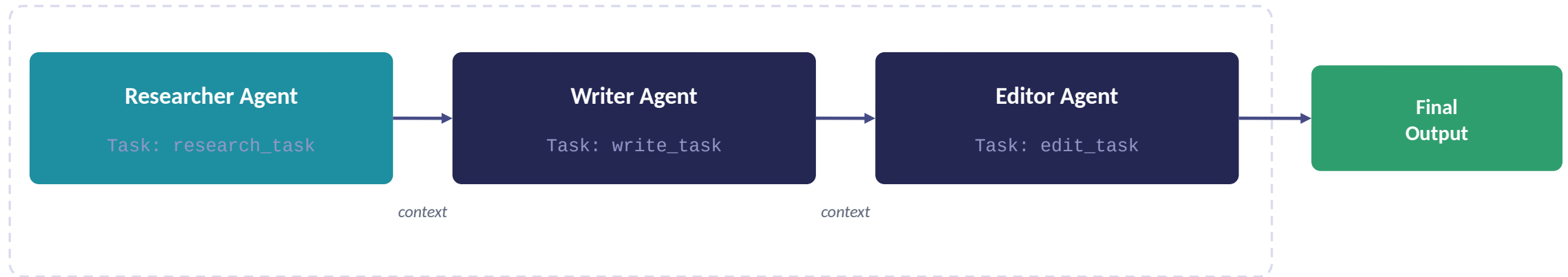
A Crew for a competitive-analysis report has a Researcher agent (role: "Senior Market Researcher") whose task is to gather pricing data, and a Writer agent (role: "Content Strategist") whose task is to turn those findings into an executive summary — the Writer's task declares the Researcher's task as context, so it receives the findings automatically.

REAL-TIME ENTERPRISE USE CASE

A weekly sales-performance report Crew runs a Data-Analyst agent (queries the data warehouse), a Writer agent (drafts the narrative), and an Editor agent (checks tone and factual consistency against the raw numbers) — under a Sequential process, producing a reviewed report with no human touching it until final sign-off.

A CrewAI Sequential Process

Crew (Process: Sequential)



Each agent's Task can declare an earlier Task as context, so the Writer's task automatically receives the Researcher's findings without any manual hand-off code.

A CrewAI Sequential Process — Explained

1. The Crew is initialized with an ordered list of agents and tasks, and `Process.sequential`.
2. `crew.kickoff()` starts the Researcher agent on `research_task` using its own tools (e.g., a web-search tool).
3. `research_task`'s output becomes available as context to any later task that declares it — here, `write_task`.
4. The Writer agent runs `write_task`, producing a draft grounded in the Researcher's findings rather than its own recall.
5. The Editor agent runs `edit_task` last, checking the draft against both the original findings and house style before releasing it.
6. The Crew returns the final task's output as the overall result — in Hierarchical mode, a manager agent would instead dynamically decide task order and could send work back for revision, mirroring Module 3's Reviewer pattern.

Building a Research + Writer Crew

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool

search_tool = SerperDevTool()

researcher = Agent(
    role="Senior Market Researcher",
    goal="Find accurate, current data on {topic}",
    backstory="A meticulous analyst who never states a claim without a source.",
    tools=[search_tool], verbose=True)

writer = Agent(
    role="Content Strategist",
    goal="Turn research findings into a clear executive summary",
    backstory="Writes concisely for busy technical leaders.", verbose=True)

research_task = Task(
    description="Research the current state of {topic}; list key findings with sources.",
    expected_output="A bulleted list of findings, each with a source citation.",
    agent=researcher)

write_task = Task(
    description="Write a 3-paragraph executive summary from the research findings.",
    expected_output="A 3-paragraph executive summary.",
    agent=writer, context=[research_task])

crew = Crew(agents=[researcher, writer], tasks=[research_task, write_task],
    process=Process.sequential)

result = crew.kickoff(inputs={"topic": "enterprise adoption of AI coding assistants"})
print(result)
```

EXPECTED OUTPUT

The Researcher produces a sourced bullet list; the Writer, receiving it as context automatically, produces a grounded 3-paragraph summary.

ENTERPRISE USE CASE

context=[research_task] is CrewAI's version of the state hand-off Module 3 built by hand and LangGraph builds via shared State.

CrewAI in Practice: Weekly Sales Report Crew

▪ Business Problem:

A regional sales team spends several hours every Monday manually pulling numbers and writing a narrative performance report for leadership.

▪ AI Solution:

A CrewAI Sequential crew of a Data-Analyst agent, a Writer agent, and an Editor agent produces a reviewed draft report automatically every Monday morning.

▪ Architecture:

Scheduled trigger → Data-Analyst Agent (query task, DB tool) → Writer Agent (narrative task, context = analyst findings) → Editor Agent (consistency-check task) → Final Output distributed to leadership.

▪ Frameworks Used:

CrewAI (Agent, Task with context chaining, Crew, Process.sequential).

EXAMPLE SCENARIO

```
"Summarize last week's regional sales
performance
vs. target, highlighting any region more than
10% off plan."
```

EXPECTED RESULT

A reviewed three-section report (numbers, narrative, flagged exceptions) lands in the leadership channel before the Monday morning meeting, with the underlying query results attached for verification.

SAFETY CONSIDERATIONS

The Data-Analyst agent's DB tool is read-only and scoped to the sales schema; the Editor agent's task explicitly requires flagging (not silently correcting) any number in the draft that doesn't match the raw query output.

MODULE 7

Framework Comparison & Selection

LEARNING OBJECTIVES

- Compare LangChain, LangGraph, and CrewAI across control model, state management, and operational maturity
- Choose the right framework for a given agentic workload using a repeatable decision process
- Contrast loop-based (open-ended) agents with safety-bounded (graph- or role-constrained) agents
- Build a web-research agent and reason about how its implementation would differ across frameworks

LangChain vs. LangGraph vs. CrewAI

Dimension	LangChain	LangGraph	CrewAI
Control model	AgentExecutor's implicit loop	Explicit state graph you define	Role-based Crew process
Best for	Single-agent tool-calling apps	Branching, cycles, durable pauses	Persona-driven multi-agent work
State handling	Conversation buffer / memory	Typed State threaded through nodes	Task context chaining
Debuggability	verbose AgentExecutor logs	Inspectable, enumerable graph paths	Per-agent/per-task trace
Human-in-the-loop	Manual, via custom callback	Native interrupt + checkpointer	Manual, via Hierarchical review
Learning curve	Low — closest to plain function calls	Moderate — graph mental model	Low — role/task mental model

None of these are mutually exclusive in production — Module 7's use case shows LangChain and LangGraph coexisting behind one API.

Loop-Based vs. Safety-Bounded Agents

DEFINITION

A loop-based agent (a plain `AgentExecutor` loop) is only as safe as the caps placed around it — typically a raw `max_iterations` count that eventually cuts off unproductive looping, but says nothing about which paths were tried. A safety-bounded agent uses a control structure — a graph's fixed set of named nodes and edges, or a Crew's fixed task list — where the set of possible next steps is enumerable in advance, not just capped after the fact.

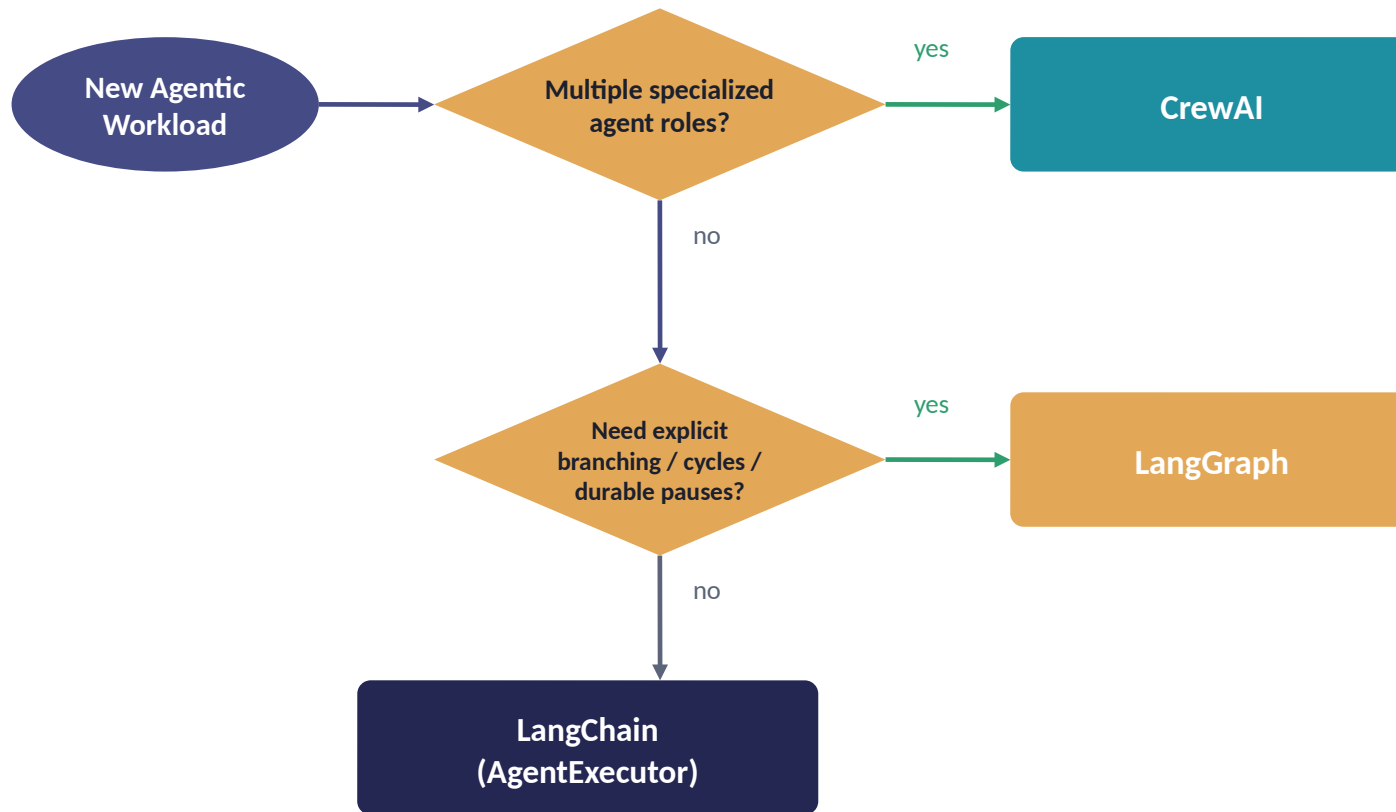
EXAMPLE

A LangChain `AgentExecutor` with `max_iterations=15` can still spend all 15 iterations retrying the same failing tool call before giving up. A LangGraph graph with a `retry_count` field in state and a conditional edge that routes to a fallback node after 2 failures bounds the identical situation structurally — the graph cannot even represent a 15-attempt retry storm.

REAL-TIME ENTERPRISE USE CASE

A support agent kept re-trying the same failing inventory-lookup call until its iteration cap cut it off, burning latency on every ticket that hit a flaky downstream API. Rebuilding its control flow as a LangGraph graph with an explicit retry-then-escalate edge fixed the behavior by removing the bad path from the graph, not by tuning a cap.

Choosing a Framework: A Decision Guide



A workload needing BOTH multiple roles and explicit graph control is not a dead end — Module 7's use case wraps a LangGraph workflow around role-scoped nodes.

Most production workloads land on the "no / no" path — a single well-scoped LangChain agent is usually simpler to operate than a multi-agent Crew or a hand-built graph.

Choosing a Framework: A Decision Guide — Explained

1. Start from the workload, not a framework preference — ask what the task structurally needs, not which library the team already knows.
2. First question: does the work naturally split into multiple specialized personas handing work to one another (research → write → edit)? If so, CrewAI's role/task/Crew model maps onto that directly.
3. If it's fundamentally one agent's job, ask the second question: does it need explicit branching, cycles, or a durable pause (a human-approval step that can resume hours or days later)?
4. If yes, LangGraph's state graph gives you an enumerable, inspectable control flow and native checkpointing for that pause — Module 5's change-approval workflow is the canonical example.
5. If no — a single agent, no need to pause and resume across separate calls — LangChain's AgentExecutor is the simplest tool that does the job, with the lowest operational overhead.
6. These aren't permanent choices: Module 7's enterprise use case shows a team starting on LangChain and migrating only the one workflow that grew a durable-pause requirement to LangGraph.

Hands-On: A Web-Research Agent

```

from langchain.agents import create_react_agent, AgentExecutor
from langchain_core.tools import tool
from langchain import hub
from langchain_openai import ChatOpenAI

@tool
def web_search(query: str) -> str:
    """Search the public web and return the top result snippets."""
    return search_client.search(query, num_results=3)

@tool
def summarize(text: str) -> str:
    """Summarize a block of text in 2-3 sentences."""
    return llm.invoke(f"Summarize concisely:\n\n{text}").content

tools = [web_search, summarize]
llm = ChatOpenAI(model="gpt-4o", temperature=0)
prompt = hub.pull("hwchase17/react") # standard ReAct prompt, Module 2

agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, max_iterations=6, verbose=True)

result = executor.invoke({"input":
    "What are the latest announcements about Oracle Database 23ai AI Vector Search?"})
print(result["output"])

```

EXPECTED OUTPUT

The agent calls `web_search`, then `summarize` on the top snippets, and returns a 2-3 sentence answer — `max_iterations=6` is the loop-based safety net from this module's concept slide.

ENTERPRISE USE CASE

Built with LangGraph: an explicit search → summarize state graph, two fixed nodes. Built with CrewAI: a Researcher agent hands findings to a Writer agent via `context=[...]` (Module 6). Same task, three different control-flow guarantees — the decision guide is what tells you which extra structure is worth paying for.

Framework Selection in Practice: A Platform Team's Migration

▪ Business Problem:

A platform team's LangChain order-support agent worked well until product asked for a durable, multi-step human-approval pause — something AgentExecutor can't do natively.

▪ AI Solution:

LangChain stays for the working single-agent tools (Module 4's SQL assistant); only the new approval workflow is rebuilt in LangGraph, using its checkpoint for the durable pause.

▪ Architecture:

Existing LangChain agents unchanged → new approval workflow as a LangGraph graph with an interrupt node → both exposed behind one internal API gateway.

▪ Frameworks Used:

LangChain (unchanged, existing agents) + LangGraph (new workflow) — chosen per Module 7's decision guide.

EXAMPLE SCENARIO

```
"Migrate only the workflows that need durable
pauses — leave working AgentExecutor agents
alone."
```

EXPECTED RESULT

Two frameworks coexist in production behind one API; the team avoided both extremes — rewriting everything into one framework, and bolting ad-hoc pause/resume logic onto AgentExecutor.

SAFETY CONSIDERATIONS

Mixing frameworks is scoped explicitly per workflow and documented in an internal architecture decision record, so on-call engineers always know which control model they're debugging before they start.

MODULE 8

Real-World Use Cases, Safety & Deployment

LEARNING OBJECTIVES

- Map agentic architecture patterns onto three common enterprise use cases: support, coding, and automation agents
- Apply structural safety and guardrail techniques appropriate to autonomous, tool-using systems
- Design a deployment architecture for an agent behind a production API
- Package and serve an agent through FastAPI, Docker, and a Streamlit front-end

Support Agents

▪ Business Problem:

Tier-1 support tickets (password resets, order status, simple returns) consume analyst time that could go to real escalations.

▪ AI Solution:

A tool-using support agent resolves common ticket categories autonomously and escalates only what it can't resolve or isn't authorized to do.

▪ Architecture:

Ticket intake → Agent (order/account/KB tools) → resolved: auto-reply, or uncertain/high-risk: hand off to a human queue with the full transcript attached.

▪ Frameworks Used:

LangChain AgentExecutor (Module 4) for the common path; a LangGraph conditional edge routes low-confidence or high-risk tickets to a human node instead of forcing a bad autonomous answer.

EXAMPLE SCENARIO

"My order hasn't arrived and it's been 10 days —
what's going on?"

EXPECTED RESULT

A large share of Tier-1 volume resolved without a human touch; the remainder escalates with full context instead of a cold handoff.

SAFETY CONSIDERATIONS

The agent's tools are read-only except a narrowly-scoped 'issue credit under \$50' action; anything larger routes to a human — Module 2's IAM-scoped tool pattern.

Coding Agents

▪ Business Problem:

Routine but time-consuming engineering tasks — dependency bumps, test scaffolding, boilerplate endpoints — queue up behind higher-value work.

▪ AI Solution:

A coding agent with repo read/write tools, a test-runner tool, and a PR-creation tool drafts the change, runs tests itself, and opens a PR only once tests pass.

▪ Architecture:

Task description → Agent (file, shell/test-runner, git tools) → iterative edit-test loop → PR opened for human review — never auto-merged.

▪ Frameworks Used:

A LangGraph graph bounds the edit-test loop to a fixed retry count via a state field, avoiding the unbounded-loop risk from Module 7.

EXAMPLE SCENARIO

```
"Bump the requests library to the latest  
2.x release and fix anything that breaks."
```

EXPECTED RESULT

A draft PR with the dependency bump, any necessary fixes, and a passing test run attached — a human still reviews and merges.

SAFETY CONSIDERATIONS

The agent's git tool can push only to an agent/-prefixed branch, never main, and its shell tool runs sandboxed with no network beyond the package registry — bounding blast radius structurally.

Automation Agents

▪ Business Problem:

Cross-system operational workflows — reconciling a vendor invoice against a PO and a receipt across three systems — are manual, slow, and error-prone.

▪ AI Solution:

A CrewAI-style multi-agent automation runs the reconciliation across systems and produces an auditable discrepancy report.

▪ Architecture:

Scheduled trigger → Data-Gathering Agent (ERP/procurement/receiving tools) → Reconciliation Agent (matches records, flags mismatches) → Report Agent → routed to the AP team.

▪ Frameworks Used:

CrewAI Sequential process (Module 6), each agent scoped to read-only tools over exactly one system.

EXAMPLE SCENARIO

"Reconcile all invoices received this week against POs and receiving records."

EXPECTED RESULT

A daily discrepancy report with auto-matched items and flagged exceptions requiring human sign-off — less manual effort, human kept in the loop for anything that doesn't match cleanly.

SAFETY CONSIDERATIONS

No agent in the crew has write access to the financial system of record; the crew's output is a report, and posting/approval is always a separate, human-triggered step.

Safety & Guardrails for Agentic Systems

DEFINITION

Guardrails are the layered controls that bound what an autonomous agent can actually do, independent of what the model decides to attempt: scoped, least-privilege tool credentials; schema-checked tool arguments and content filters on generated output; human-in-the-loop checkpoints for high-risk actions; iteration and cost caps; and full transcript logging for audit. The model's own good judgment is never the only safety mechanism — every IAM-scoped tool, human-approval node, and bounded loop in this course has been an instance of this.

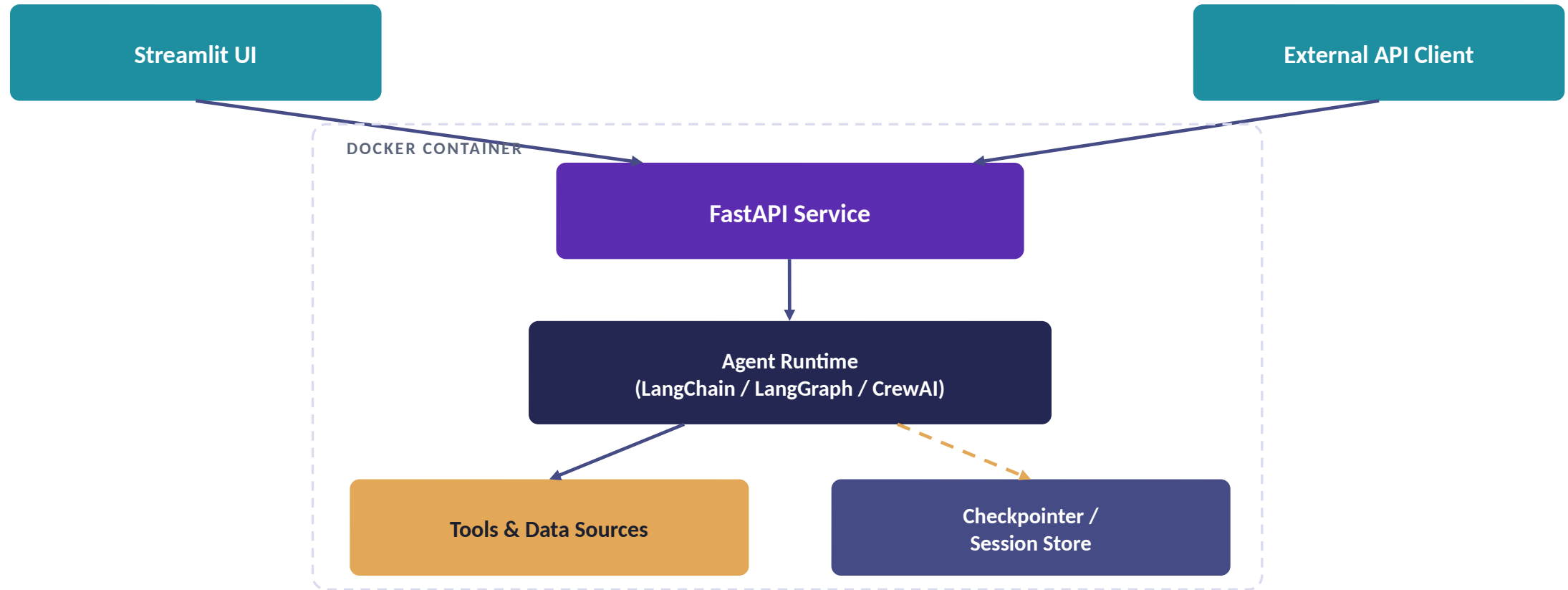
EXAMPLE

A refund-issuing tool isn't 'the agent promises not to over-refund' — it's a database role with a hard \$50 cap enforced server-side, a required ticket_id foreign-key check, and a nightly reconciliation job that alerts if any tool call bypassed the application layer entirely.

REAL-TIME ENTERPRISE USE CASE

An enterprise agent platform team maintains a shared guardrail library — pre-built scoped-credential patterns, a standard LangGraph human-approval node, a standard confidence-threshold router — so every new agent team inherits the same safety posture instead of re-deriving it from scratch.

Deployment Architecture: API, Docker & Streamlit



The same Docker image runs identically behind the Streamlit demo UI and in production — no drift between what was tested in CI and what actually ships.

Deployment Architecture: API, Docker & Streamlit — Explained

1. A request arrives either through a Streamlit UI (for interactive or demo use) or directly from another internal service calling the API — both are just HTTP clients to the same backend.
2. FastAPI exposes a single `/invoke` endpoint that validates the request against a Pydantic schema and hands it to the agent runtime — this is the one place authentication and rate-limiting are enforced.
3. The Agent Runtime is whichever framework the workload calls for (Module 7's decision guide): a LangChain AgentExecutor, a compiled LangGraph graph, or a CrewAI Crew — the API layer doesn't care which.
4. The agent calls out to Tools & Data Sources to ground its reasoning in real systems, exactly as in every hands-on lab in this course.
5. For a LangGraph-based runtime, a Checkpointer / Session Store persists state between calls — what makes a durable human-in-the-loop pause (Module 5) possible across separate HTTP requests.
6. The whole FastAPI + Agent Runtime + Tools boundary ships as one Docker image, so the container that passed CI is exactly what runs in every environment.

Deploying an Agent Behind FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel
import uvicorn

app = FastAPI(title="Agentic AI Service")

class InvokeRequest(BaseModel):
    input: str
    thread_id: str | None = None

@app.post("/invoke")
def invoke_agent(req: InvokeRequest):
    config = {"configurable": {"thread_id": req.thread_id or "default"}}
    result = compiled_graph.invoke(
        {"messages": [("user", req.input)]}, config=config)
    return {"output": result["messages"][-1].content}

@app.get("/healthz")
def healthz():
    return {"status": "ok"}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

EXPECTED OUTPUT

POST /invoke {"input": "...", "thread_id": "session-42"} returns {"output": "..."}
 — the same thread_id resumes a paused human-in-the-loop graph from
 Module 5 across separate HTTP calls.

ENTERPRISE USE CASE

This FastAPI service is the one artifact that gets Dockerized (FROM python:3.12-slim, pip install, CMD) and deployed identically whether it's called by the Streamlit demo or another internal service — this module's diagram, made concrete.

MODULE 9

Capstone Project: Autonomous Research Assistant

LEARNING OBJECTIVES

- Design an end-to-end agentic system that combines planning, tool use, memory, and multi-step reasoning
- Select and justify a framework (or combination) using this course's decision guide
- Apply structural safety guardrails and a deployment plan to a complete, portfolio-ready project
- Present a working architecture that could be defended in an Oracle enterprise design review

Capstone: The Autonomous Research Assistant

OBJECTIVE *Build an Autonomous Research Assistant that takes a research question, plans sub-investigations, gathers grounded evidence via tools, synthesizes a cited report, and pauses for human approval before publishing.*

- **Scenario:**

An Oracle product-strategy team needs a recurring competitive-intelligence brief. Manually, an analyst spends half a day per topic gathering and synthesizing sources; the team wants that cut to under an hour of human review time.

- **Steps:**

1) Plan: decompose a research question into 3-5 sub-questions. 2) Research: a Solver agent calls a web-search tool and a summarize tool per sub-question. 3) Synthesize: combine findings into a draft report. 4) Review: check source coverage and recency (Module 3's Planner/Solver/Reviewer pattern). 5) Express the pipeline as a LangGraph state graph with a human-approval interrupt before release (Module 5), served behind the FastAPI + Streamlit stack from Module 8.

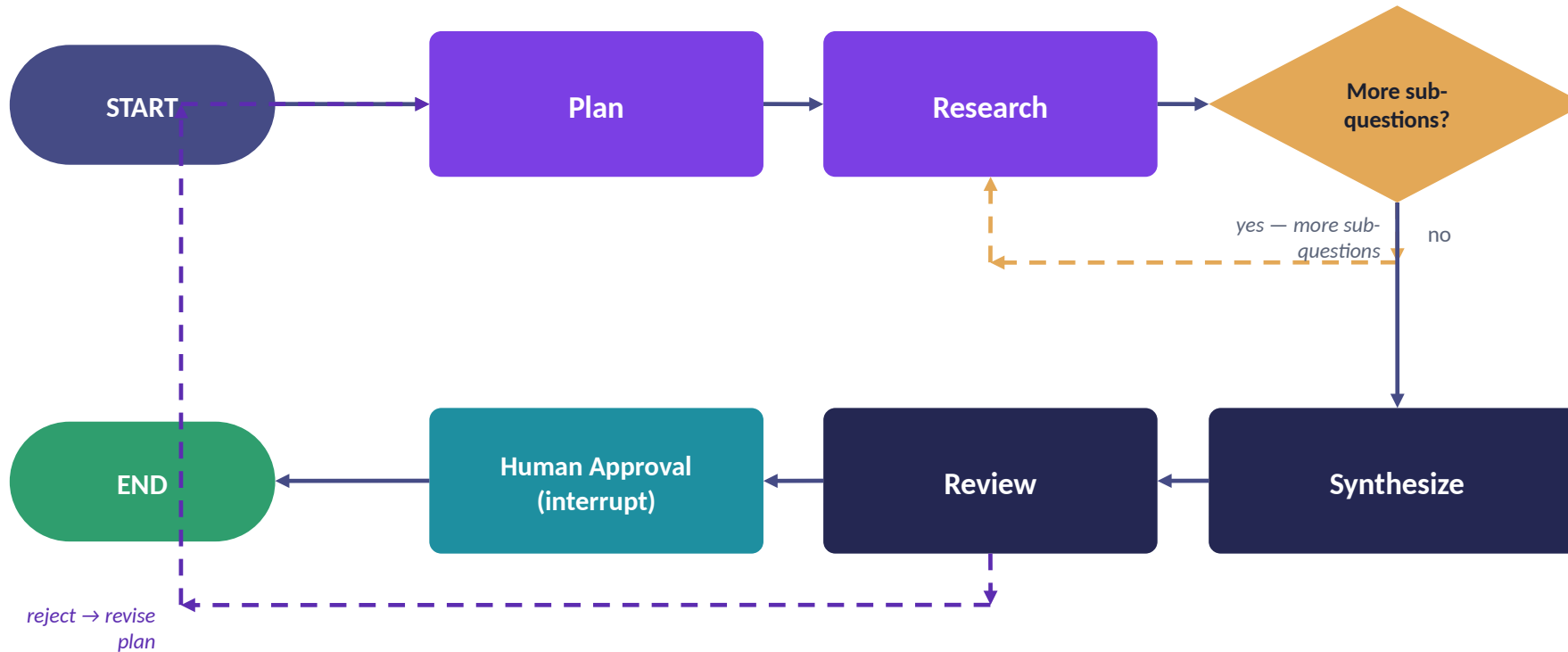
- **Deliverable:**

A working LangGraph application plus a one-page architecture diagram and a short write-up justifying every framework choice against Module 7's decision guide — ready to defend in a design review.

- **Frameworks / Tools Used:**

LangGraph (StateGraph, conditional edges, interrupt, checkpointner); a web-search tool and a summarize tool (Modules 2 & 7); the Planner/Solver/Reviewer role pattern (Module 3); FastAPI + Streamlit for deployment (Module 8).

Capstone Architecture: The Research-Assistant Graph



Every node here is a pattern from an earlier module: Plan/Research/Synthesize/Review is Module 3's role split; the conditional-edge loop and the human-approval interrupt are Module 5's LangGraph patterns.

Capstone Architecture: The Research-Assistant Graph — Explained

1. Plan decomposes the incoming research question into 3-5 concrete sub-questions — the same Planner role from Module 3, now a LangGraph node.
2. Research runs one sub-question at a time through a search-and-summarize tool pair (Module 2/7's pattern), appending grounded findings to shared state.
3. A conditional edge checks whether sub-questions remain: if so, it loops back to Research — an explicit, bounded cycle, not an open-ended AgentExecutor loop (Module 7).
4. Once every sub-question is answered, Synthesize drafts a single cited report from the accumulated findings.
5. Review checks source coverage and recency; if it rejects the draft, a loop-back routes to Plan with feedback, mirroring Module 3's Planner/Solver/Reviewer revise cycle.
6. Once Review passes, the graph pauses at Human Approval — a real interrupt, resumable hours later via the same thread_id (Module 5) — before finally routing to END and publishing the report.

Capstone Starter: The Research-Assistant State Graph

```
# imports as in Module 5: TypedDict, Annotated, operator, StateGraph, START, END, MemorySaver
class ResearchState(TypedDict):
    question: str; subquestions: list[str]; report: str; approved: bool
    findings: Annotated[list[str], operator.add]

def plan(state): return {"subquestions": decompose(state["question"])}
def research(state):
    sq = state["subquestions"].pop(0)
    return {"findings": [summarize(web_search(sq))]}
def more_subquestions(state):
    return "research" if state["subquestions"] else "synthesize"
def synthesize(state): return {"report": write_report(state["findings"])}
def review(state): return {"approved": check_coverage(state["report"])}
def route_after_review(state):
    return "human_approval" if state["approved"] else "plan"

graph = StateGraph(ResearchState)
for name, fn in [("plan", plan), ("research", research), ("synthesize", synthesize),
                ("review", review), ("human_approval", lambda s: s)]:
    graph.add_node(name, fn)
graph.add_edge(START, "plan")
graph.add_edge("plan", "research")
graph.add_conditional_edges("research", more_subquestions,
    {"research": "research", "synthesize": "synthesize"})
graph.add_edge("synthesize", "review")
graph.add_conditional_edges("review", route_after_review,
    {"human_approval": "human_approval", "plan": "plan"})
graph.add_edge("human_approval", END)
app = graph.compile(checkpointer=MemorySaver(), interrupt_before=["human_approval"])
```

EXPECTED OUTPUT

The graph pauses at `human_approval` (via `interrupt_before`) once a draft passes review — a human calls `app.invoke(None, config)` with the same `thread_id` to resume and publish.

ENTERPRISE USE CASE

Every piece here already exists: Plan/Research/Synthesize/Review mirrors Module 3's roles, the loop mirrors Module 5's ReAct graph — the capstone composes known patterns, it doesn't invent new ones.

Key Takeaways

- Agentic AI is a control loop — Reason → Act → Observe — not a bigger prompt (Module 1).
- ReAct, CoT/ToT/GoT, tool calling, and reflection are the reasoning primitives every framework builds on (Module 2).
- Memory, RAG, and the Planner/Solver/Reviewer pattern ground and coordinate multi-step work (Module 3).
- LangChain, LangGraph, and CrewAI differ in control model, state handling, and how explicitly they bound the agent (Modules 4–6).
- Framework choice is a design decision — driven by roles, branching, and durability needs, not default habit (Module 7).
- Safety comes from structural guardrails — scoped credentials, bounded loops, human-approval nodes — never from asking nicely (Module 8).
- The capstone composes every pattern in this course into one system — that composability is the real skill (Module 9).

Hands-On Exercises Recap

Module	Hands-On Exercise
1	A Minimal Calculator Agent
2	A Production ReAct Prompt Template • Implement a ReAct Prompt Template
3	A Planner-Worker System
4	A Tool-Calling LangChain Agent
5	A LangGraph State Graph with Checkpointing
6	A CrewAI Research + Writer Crew
7	A Web-Research Agent, Framework-Compared
8	An Agent Deployed Behind FastAPI
9	Capstone: The Autonomous Research Assistant

THANK YOU

Agentic AI

Engineering, Frameworks & Enterprise Deployment — LangChain · LangGraph · CrewAI

Questions & Discussion